Machine Dependent Assembler features :

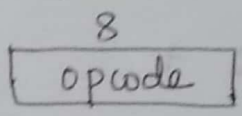
Here, we consider an example of SIC/XE machine, as we already know, SIC/XE has

a) Registers : A X L B S T F PC SW
(0 1 2 3 4 5 6 8 9)

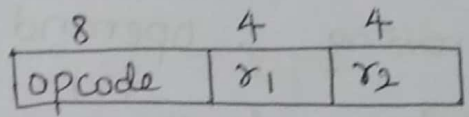
b) Data format : Integer : 3 bytes
Character : 1 byte
Float : 6 byte.

c) Instruction Formats :

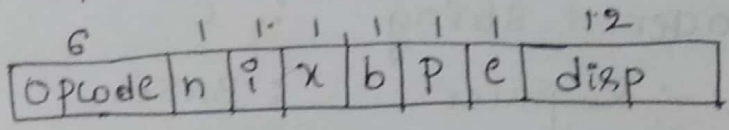
Format 1 : 1 byte



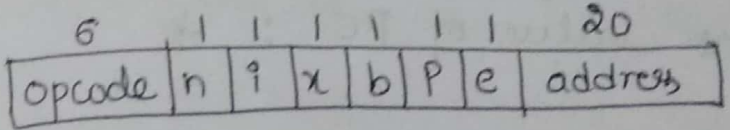
Format 2 : 2 byte



Format 3 : 3 byte



Format 4 : 4 byte



d) Addressing modes are determined based on 6 bits.

n	i	x	Addressing mode
1	0		Indirect addressing
0	1		Immediate
1	1		Not immediate, Not indirect
0	0		Simple addressing
		1	Indexed addressing
		0	Direct addressing

ii)

b	p	e	Addressing Modes
0	1		Program counter Relative
1	0		Base relative
1	1		Invalid
0	0		No PC relative, No base relative
		1	Format 4 instruction
		0	Format 3 instruction.

Different Addressing mode notations:

1) Indirect Addressing : @
2) Immediate Addressing : #
3) Extended Format : +
4) Indexed Addressing : operand, X
5) character string : C ' '
6) Base - Register : BASE
7) Current value of PC : *

The addressing priority are as follows.

a) pc relative addressing : $-2048 \leq disp \leq 2047$

b) Base relative addressing : $0 \leq disp \leq 4095$

Note: Negative numbers are represented in 2's complement

Procedure to create object code for SIC/XE.

1) write the LOCCTR address for each instruction in the program.

↳ if operand field is

i) Memory address → format 3 ⇒ Add 3 bytes
ii) Register – Register → format 2 ⇒ Add 2 bytes
iii) + before ~~operand~~ opcode. → format 4 ⇒ Add 4 bytes.

→ if it is RESW 2000.

2000×3 bytes $= (6000)_d = (1770)_H$

Add these many bytes to address.

→ RESW 1 ⇒ Add just 3 bytes.

→ RESB 2000 ⇒ Add 2000 bytes in the Hexadecimal

i.e $(2000)_d = (700)_H$

→ RESB 4096

$(4096)_d = (1000)_H$ ⇒ Add 1000 bytes

→ BYTE C 'EOF' ⇒ count the length of constant and add those many bytes.

→ Enter the labels onto SYMTAB (pass 1).

→» Once we are done with LOCCTR calculation then finding program length = End Address — START Address

3) Now start creating the object code (pass 2) based on different Addressing modes and set corresponding bits and calculate displacement.

→ for Extended format, displacement = address

→ for Reg to Reg instruction, write the opcode address followed by register number.

Ex: clear X ⇒ B410 (format 2)

→ for PC relative, disp = TA − PC

→ for Base relative, disp = TA − (B)

Consider the SIC/XE machines examples.

Line	LOC		Source statement		Object code
5	0000	COPY	START	0	
10	0000	FIRST	STL	RETADR	17202D
12	0003		LDB	#LENGTH	69202D
13			BASE	LENGTH	
15	0006	CLOOP	+JSUB	RDREC	4B101036
20	000A		LDA	LENGTH	032026
25	000D		COMP	#0	290000
30	0010		JEQ	ENDFIL	332007
35	0013		+JSUB	WRREC	4B10105D

Line	Loc		Source Statement		Object code
40	0017	3	J	CLOOP	3F2FEC
45	001A	3ENDFIL	LDA	EOF	032010
50	001D	3	STA	BUFFER	0F2016
55	0020	3	LDA	#3	010003
60	0023	3	STA	LENGTH	0F200D
65	0026	4	+JSUB	WRREC	4B10105D
70	002A	3	J	@RETADR	3E2003
80	002D	3EOF	BYTE	C'EOF'	454F46
95	0030	3RETADR	RESW	1 × 3	
100	0033	3 LENGTH	RESW	1 × 3	
105	0036	1000 BUFFER	RESB	4096. × 1 = 1000	
110					
115					
120					
125	1036	2RDREC	CLEAR	X	B410
130	1038	2	CLEAR	A	B400
132	103A	2	CLEAR	S	B440
133	103C	4	+LDT	#4096	75101000
135	1040	3 RLOOP	TD	INPUT	E32019
140	1043	3	JEQ	RLOOP	332FFA
145	1046	3	RD	INPUT	DB2013
150	1049	2	COMPR	A, S	A004
155	104B	3	JEQ	EXIT	332008
160	104E	3	STCH	BUFFER, X	57C003
165	1051	2	TIXR	T	B850
170	1053	3	JLT	RLOOP	3B2FEA
175	1056	3EXIT	STX	LENGTH	134000
180	1059	3	RSUB		4F0000
185	105C	INPUT	BYTE	X'F1'	F1

```
195
200
205
210   105D  2  WRREC   CLEAR   X            B410
212   105F  3           LDT     LENGTH       774000
215   1062  3  WLOOP    TD      OUTPUT       E32011
220   1065  3           JEQ     WLOOP        332FFA
225   1068  3           LDCH    BUFFER,X     53C003
230   106B  3           WD      OUTPUT       DF2008
235   106E  2           TIXR    T            B850
240   1070  3           JLT     WLOOP        3B2FEF
245   1073  3           RSUB                 4F0000
250   1076  1  OUTPUT   BYTE    X'05'        05
255   1077             END     FIRST
```

Consider the above example,

1> Add the length of each instruction and add it to the LOCCTR and find the program length.

Program Length = End address − Start address

= 1077 − 0000 = 1077.

2> Create the symbol table.

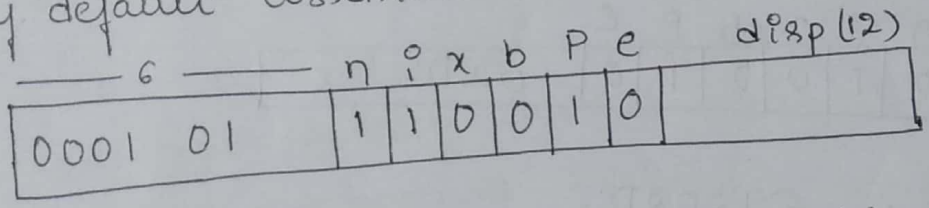
Symbol Name	PC Value
FIRST	0000
CLOOP	0006
ENDFIL	001A
EOF	002D
RETADR	0030
LENGTH	0033
BUFFER	0036
RDREC	1036

Symbol table	PC value
RLOOP	1040
EXIT	1056
INPUT	105C
WRREC	105D
WLOOP	1062
OUTPUT	1076

3> Start creating object code. (pass-2)

10 0000 FIRST STL RETADR

By default assembler uses PC relative addressing.



STL = 14

RETADR = 0030

TA = PC + disp

disp = TA − PC

= RETADR − LOCCTR

= 0030 − 0003

= 002D

→ opcode value is 6 bits

Ex: STL = 14 = 000101 ~~00~~ (last 2 bits discarded)

if it is 6 ⇒ 11 ~~00~~

→ The above instruction is not immediate, not indirect so set n=1, i=1.

→ The above instruction is not indexed and not ~~Base~~ PC relative ∴ x=0, b=0. but PC relative so P=1. not a format 4 ∴ e=0.

∴ Object code is.

1	01	1	1	0	0	1	0	02D

∴ 172 02D.

---

12 0003 LDB #LENGTH

The above instruction is immediate, ∴ i=1, n=0

PC instruction PC=1,

TA = PC + disp

disp = TA − PC = 0033 − 0006 = ϕ 02D

	n	i	x	b	p	e	
0110 10	0	1	0	0	1	0	02D.

opcode ∴ 69202D.

LDB = 68

---

15 0006 CLOOP +JSUB RDREC

The above instruction is format 4

∴ disp = 20 bits.

Not immediate and not indirect n=1, i=1

opcode							
000010	1	1	0	0	0	0	01036

∴ 4B101036

---

20. 000A LDA LENGTH → format 3

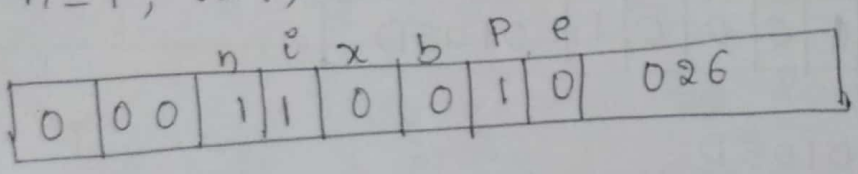
→ pc relative, disp = TA − PC

= 0033 − 000D

= 026.

→ Not immediate, not indirect, not indexed so

n=1, i=1, x=0



∴ ⇒ 032026.

---

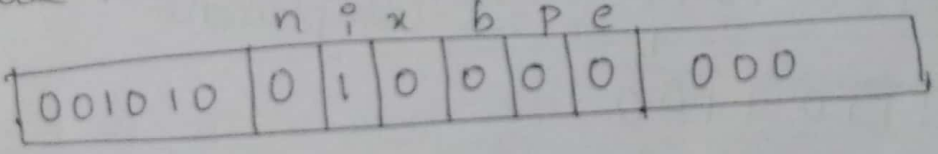
85 000D comp #0

→ Not pc relative because operand is direct value but not memory address.

∴ disp = operand = 000

→ immediate addressing n=0, i=1, b=0, p=0

→ opcode value → comp = 28.



⇒ 290000

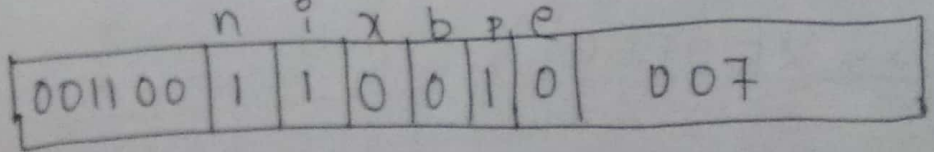
---

30 0010 JEQ ENDFIL ⇒ format 3

pc relative ∴ disp = TA − PC

= 001A − 0013 = 007

not immediate, not indirect n=1, i=1



JEQ = 30 ⇒ 332007

35. 0013 +JSUB NRREG

→ format 4

Disp = addr of operand

= 0105D

→ JSUB = 48

n	i	x	b	p	e	
010010	1	1	0	0	0	1	0105D

⇒ 4B10105D.

---

40. 0017 J CLOOP ⇒ format 3

disp = TA − PC

= 0006 − 001A

= −14 (it takes 2's complement in 12 bit field

−14 = 0000 0001 0100

→ 1's complement

1111 1110 1011

1 → 2's complement

1111 1110 1100

(F E C)

→ It is PC relative, not immediate, not indirect

n=1, i=1

n	i	x	b	p	e	
0011 11	1	1	0	0	1	0	FEC

opcode → J = 3C CLOOP = 0006

∴ ⇒ 3F2FEC

45 '001A ENDFIL LDA EOF ⇒ format 3 (PC relative)
00 002D

disp = TA − PC

= 002D − 001D = ∅010

n	i	x	b	p	e	
000000	1	1	0	0	1	0	010

⇒ 032010

---

50. 001D STA BUFFER ⇒ format 3 (PC relative)
0C 0036

disp = TA − PC = 0036 − 0020 = ∅016

n	i	x	b	p	e	
000011	1	1	0	0	1	0	016

⇒ 0F2016

---

55. 0020 LDA #3 ⇒ immediate address.
00

n	i	x	b	p	e	
000000	0	1	0	0	0	0	003

⇒ 010003

---

60. 0023 STA LENGTH ⇒ format 3 (PC relative)
0C 0033

disp = TA − PC = 0033 − 0026 = ∅00D

n	i	x	b	p	e	
000001	1	1	0	0	1	0	00D

⇒ 0F200D.

---

65. 0026 +JSUB WRREC → format 4
48 105D

disp = address of operand = 105D

n	i	x	b	p	e	
000010	1	1	0	0	0	1	0105D

⇒ 4B10105D

70. 002A J @RETADR → format 3, indirect, PC

3C 0030

	n	i	x	b	P	e	
001111	1	0	0	0	1	0	003

disp = TA − PC

= 0030 − 002D = ∅003

⇒ 3E2003

---

80. 002D EOF BYTE C'EOF'.

→ Convert EOF to hexadecimal ascii value

E → 45 ⇒ 454F46

O → 4F

F → 46.

---

125. 1036 RDREC CLEAR X

B4

The above instruction is Register-to-register mode

⇒ B410

→ only 2 bytes ~~since it is~~

---

130 1038 CLEAR A ⇒ B400

---

132 103A CLEAR S ⇒ B440

---

133 103C +LDT #4096

74

→ format 4 & immediate addressing

disp=(4096) = 01000

	n	i	x	b	P	e	
011101	0	1	0	0	0	1	01000

⇒ 75101000

135. 1040 RLOOP TD INPUT

E0 105C

→ format 3 + PC relative

disp = TA − PC = 105C − 1043 = 019

n	i	x	b	p	e	
1	1	0	0	1	0	019

1110 00

⇒ E32019

140. 1043 JEQ RLOOP ⇒ format 3 + PC relative

disp = TA − PC = 1040 − 1046 = −6

−6 ⇒ 2's complement

0000 0000 0110 → 1's

1111 1111 1001

1

1111 1111 1010 ⇒ FFA

n	i	x	b	p	e	
1	1	0	0	1	0	FFA

0011 00

⇒ 332FFA

145. 1046 RD INPUT ⇒ Format 3 + pc relative

D8 105C

disp = TA − Pc = 105C − 1049 = 013

n	i	x	b	p	e	
1	1	0	0	1	0	013

110110

⇒ DB2013

150. 1049 COMPR A, S ⇒ A004

155 104B JEQ EXIT ⇒ format 3 + PC relative

30 1056

disp = TA − PC = 1056 − 104E = 008

	n	i	x	b	p	e	
001100	1	1	0	0	1	0	008

⇒ 332008.

160. 104E STCH BUFFER, X

54 0036

The length is stored in base register at 0033.

∴ disp = TA − B

= 0036 − 0033 = ø003

	n	i	x	b	p	e	
010101	1	1	1	1	0	0	003

⇒ 57C003.

165. 1051 TIXR T ⇒ B850

170. 1053 JLT RLOOP ⇒ format 3 + PC relative

38 1040

disp = TA − PC = 1040 − 1056 = FEA

	n	i	x	b	p	e	
001110	1	1	0	0	1	0	FEA

⇒ 3B2FEA

175. 1056 EXIT STX LENGTH ⇒ format 3 + PC relative

10 0033

disp = TA − PC = 0033 − 1059 = EFDA > 2047

∴ go for base relative mode

disp = TA − B = 0033 − 0033 = ø000

	n	i	x	b	p	e	
000100	1	1	0	1	0	0	000

⇒ 134000

180. 1059 RSUB ⇒ format 3

4C

n	i	x	b	p	e	
010011	1	1	0	0	0	0	000

⇒ 4F0000

185. 105C INPUT BYTE X'F1'

⇒ character string, store as it is.

210. 105D WRREC CLEAR X

⇒ B410

212. 105F LDT LENGTH ⇒ format 3

74 0033

disp = TA − PC = 0033 − 1062 = EFD1 > 2047

∴ go for base relative

disp = TA − (B) = 0033 − 0033 = 0000

n	i	x	b	p	e	
011101	1	1	0	1	0	0	000

⇒ 774000

215. 1062 WLOOP TD Output ⇒ format 3 + pc relative

E0 1076

disp = TA − PC = 1076 − 1065 = 011

n	i	x	b	p	e	
111000	1	1	0	0	1	0	011

⇒ E32011

220. 1065 JEQ WLOOP ⇒ format 3 + pc relative

30 1062

disp = TA − PC = 1062 − 1068 = FFFA

n	i	x	b	p	e	
001100	1	1	0	0	1	0	FFA

⇒ 332FFA

225. 1068 LDCH BUFFER, X ⇒ indexed.
50 0036

disp = TA − PC = 0036 − 106B = −1035 > 2047

∴ go for Base relative mode

disp = TA − B = 0036 − 0033 = ϕ003

	n	i	x	b	P	e	
010100	1	1	1	1	0	0	003

⇒ 53C003.

230. 106B WD OUTPUT ⇒ Format 3 + PC relative.
dc 1076

disp = TA − PC = 1076 − 106E = ϕ008

	n	i	x	b	P	e	
110111	1	1	0	0	1	0	008

⇒ DF2008.

235. 106E TIXR T ⇒ B850

240. 1070 JLT WLOOP ⇒ format 3 + PC relative
38 1062

disp = TA − PC = 1062 − 1073 = FEF

	n	i	x	b	P	e	
001110	1	1	0	0	1	0	FEF

⇒ 3B2FEF

245. 1073 RSUB ⇒ format 3
4C

	n	i	x	b	P	e	
010011	1	1	0	0	0	0	000

⇒ 4F0000

250. 1076 OUTPUT BYTE X'05'
object code
⇒ character stored as ~~opcode~~ value '05'

4> Object program:

H^COPY ^ 000000^001077

T^000000^1D^17202D^69202D^4B101036^032026^290000^
332007^4B10105D^3F2FEC^032010.

T^000 01D^15^0F2016^010003^0F200D^4B10105D^3E2003^
454F46^B410

T^001038^1b^B400^B440, 75101000^E32019^332FFA^
DB2013^A004^332008^57C003^B850

T^001053^1d^3B2FEA^134000^4F0000^F1^B410^774000^
E32011^332FFA^53C003^DF2008

T^00106E^09^B850^3B2FEF^4F0000^05

E^000000

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	17	20	2D	69	20	2D	4B	10	10	36	03	20	26	29	00	00
0010	33	20	07	4B	10	10	5D	3F	2F	EC	03	20	10	0F	20	16
0020	01	00	03	0F	20	0D	48	10	10	5D	3E	20	03	45	4F	46
0030	RETADR			LENGTH												
						BUFFER										
1030							B4	10	B4	00	B4	40	75	10	10	00
1040	E3	20	19	33	2F	FA	DB	20	13	A0	04	33	20	08	57	C0
1050	03	B8	50	3B	2F	EA	13	40	00	4F	00	00	F1	B4	10	77
1060	40	00	E3	20	11	33	2F	FA	53	C0	03	DF	20	08	D8	50
1070	3B	2F	EF	4F	00	00	05									

Examples :

1. Generate the complete object program for the following assembly level program.

Given, CLEAR = B4, LDA = 00, LDB = 68, ADD = 18, TIX = 2C, JLT = 38, STA = 0C

Pass I	LENGTH	LABEL	OPCODE	OPERAND	PASS-II
0000		SUM	START	0	
0000	2		CLEAR	X	B410
0002	3		LDA	#0	010000
0005	4		+LDB	#TOTAL	69101789
			BASE	TOTAL	
0009	3	LOOP	ADD	TABLE, X	1BA00D
000C	3		TIX	COUNT	2F2007
000F	3		JLT	LOOP	3B2FF7
0012	4		+STA	TOTAL	0F101789
0016	3	COUNT	RESW	1	
0019	1770	TABLE	RESW	2000	
1789	3	TOTAL	RESW	1	
178C			END	FIRST	

RESW 2000 $\Rightarrow$ $2000 \times 3 = (6000)_d$ bytes $= (1770)_H$

$\therefore$ 0019 + 1770 = 1789.

Program length = 178C − 0000 = 178C

Object code:

1> 0000 CLEAR X (Register-to-Register

⇒ directly opcode with Register numbers.

⇒ B410

2> 0002 LDA #0 ⇒ format 3, immediate addressing.

	n	i	x	b	P	e	
000000	0	1	0	0	0	0	000

⇒ 01000

3> 0005 +LDB #TOTAL ⇒ extended + immediate

	n	i	x	b	P	e	
011000	0	1	0	0	0	1	1789

disp = operand address = 01789

⇒ 69101789

4> 0009 LOOP ADD TABLE, X ⇒ indexed with PC relative

disp = TA − PC

= 0019 − 000C = 00D

	n	i	x	b	P	e	
000110	1	1	1	0	1	0	00D

⇒ 1BA00D

5> 000C TIX COUNT ⇒ format 3

disp = 0016 − 000F = 007

(COUNT) − (PC)

	n	i	x	b	P	e	
001011	1	1	0	0	1	0	007

⇒ 2F2007

6> 000F JLT Loop

38 0009

disp = TA − PC

= 0009 − 0012 = +FF7

	n	i	x	b	p	e	
001110	1	1	0	0	1	0	FF7

⇒ 3B2FF7

7> 0012 +STA TOTAL

0C 1789

	n	i	x	b	p	e	
000011	1	1	0	0	0	1	01789

⇒ 0F101789.

object program:

H^SUM---^0000^00178C

T^000000^16^B410^010000^69101789^1BA00D^2F2007^3F2FF7^0F101789

E^000000.

2. Generate the complete object program for the following assembly level program. Also indicate the contents of symbol table at the end. Assume standard SIC model and assume the following machine codes in Hex.

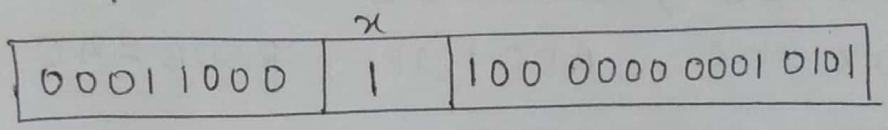
Given LDA = 00 , STA = 0C TIX = 2C JLT = 38
LDX = 04 , ADD = 18 , RSUB = 4C

Pass 1	LENGTH	LABEL	OPCODE	OPERAND	PASS-2
4000		SUM	START	4000	
4000	3	FIRST	LDX	ZERO	045788
4003	3		LDA	ZERO	005788
4006	3	LOOP	ADD	TABLE, X	18C015
4009	3		TIX	COUNT	2C5785
400C	3		JLT	Loop	384006
400F	3		STA	TOTAL	0C578B
4012	3		RSUB		4C0000
4015	1770	TABLE	RESW	2000	
5785	3	COUNT	RESW	1	
5788	3	ZERO	WORD	0	000000
578B	3	TOTAL	RESW	1	
578E		~~END~~	END	FIRST	

Program Length = 578E − 4000 = 178E

→ Since it has 2 Addressing modes
  ↳ Direct Addressing ($x=0$)
  ↳ Indexed Addressing ($x=1$)

→ 4006 LOOP ADD TABLE, X
18 4015



⇒ 18C015

Object program:

H^SUM ^004000^ 00178E

T^004000^15^045788^005788^18C015^2C5785^384006^
0C578B^4C0000

T^005788^03^000000

E^004000

3> Generate the object code for each statement in the following SIC/XE program to generate the object Program.

Given LDX = 04, LDB = 68, TIX = 2C
STA = 0C, LDA = 00, ADD = 18
JLT = 38, RSUB = 4C

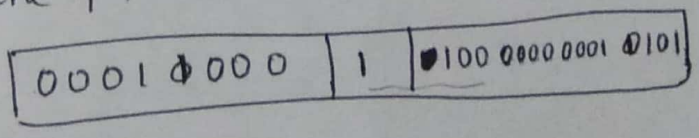
Write an object code for the following SIC/ machine.

	Loc	Len				
1	Loc	Len	Sum	START	4000	
2	4000	3	FIRST	LDX	ZERO	045788
3	4003	3		LDA	ZERO	005788
4	4006	3	Loop	ADD	TABLE,X	18C015
5	4009	3		TIX	COUNT	2C5785
6	400C	3		JLT	Loop	384006.
7	400F	3		STA	TOTAL	0C578B
8	4012	3		RSUB		4C0000
9	4015	1770	TABLE	RESW	2000×3	
10	5785	3	COUNT	RESW	1×3	
11	5788	3	ZERO	WORD	0	000000
12	578B	3	TOTAL	RESW	1	
13	578E			END	FIRST	

Given op code values are.

LDX = 04
LDA = 00
ADD = 18
TIX = 2C
JLT = 38
STA = 0C
RSUB = 4C

line 4.



18C015

object program

H^ SUM ^ 004000 ^ 001786

T^004000^15^045788^005788^...^4C0000

T^005788^3^000000

E^004000.

Generate ~~an~~ an object code & object program ~~for~~ for SIC/XE. Machine.

```
SUM     START   0
FIRST   LDX     #0
        LDA     #0
        +LDB    #TABLE2
        BASE    TABLE2
LOOP    ADD     TABLE,X
        ADD     TABLE2,X
        TIX     COUNT
        JLT     LOOP
        +STA    TOTAL
        RSUB
COUNT   RESW    1
TABLE   RESW    2000
TABLE2  RESW    2000
TOTAL   RESW    1
        END     FIRST
```

Program Relocation:

Absolute assembly program is one which executes properly, only if program is loaded from specified location.

Ex: All SIC programs are absolute assembly program.

Consider the SIC program.

```
5   1000   COPY    START   1000
10  1000   FIRST   STL RETADDR   141033
15  1003   CLOOP   JSUB RDREC    482039
        :
55  101B       .   LDA THREE     00102D
        :
85  102D   THREE   WORD 3        000003.
```

→ Here program is loaded at address 1000.
↳ The line no 55 specifies that the Reg A is to be loaded from memory address 102D (object code).
→ Suppose we attempt to load and execute the program at address 2000 instead of address 1000, the address 102D will not contain the value that we expected, as it might be a part of some other user's program.
→ Obviously we need to make some changes in the address portion of this instruction so that we can load and execute the program at address 2000.
→ at the same time there are statements like line no 85 which generates constant 3, that should remain the same regardless of where the program is loaded.

→ from the object code it is not possible to t... which values represent addresses and which represent constant data items.

→ This is all because the assembler doesnot know the actual location where the program will be loaded till the load time, therefore it cannot make the necessary changes required.

→ only parts of the program that require modification at load time are those that specify direct addresses.

→ This is achieved through relocatable program. (for SIC/XE Machines)

The program relocation is a process of modifying the addresses used in address sensitive instruction of a program. such that program can execute correctly from the allocated memory area.

It is often needed to have more than one program at same time, sharing the other resource of the machine. because of this, it is necessary to load a program into memory whenever it is available. Hence relocation of the addresses in the program is required and this will be done during loading time. Assembler only indicates those instructions which need modification and this information is passed to loader.

→ The assembler solves the relocation problem as follows which,

↳ 1) Keep track of operand address relative to start of a program.

↳ 2) Generating commands for loader which add the beginning address to operand relative address.

An object program that contains the information necessary to perform this kind of modification is called a "relocatable program". We can accomplish this with a modification record as follows.

Modification Record:

Col. 1 : M

Col. 2-7 : Starting location of the address field to be modified, relative to the beginning of the program (hexadecimal)

Col. 8-9 : Length of the address field to be modified, in half bytes (hexadecimal).

→ The length is stored in ~~bytes~~ half bytes (rather than bytes) because the address field to be modified may not occupy an integral no of bytes.

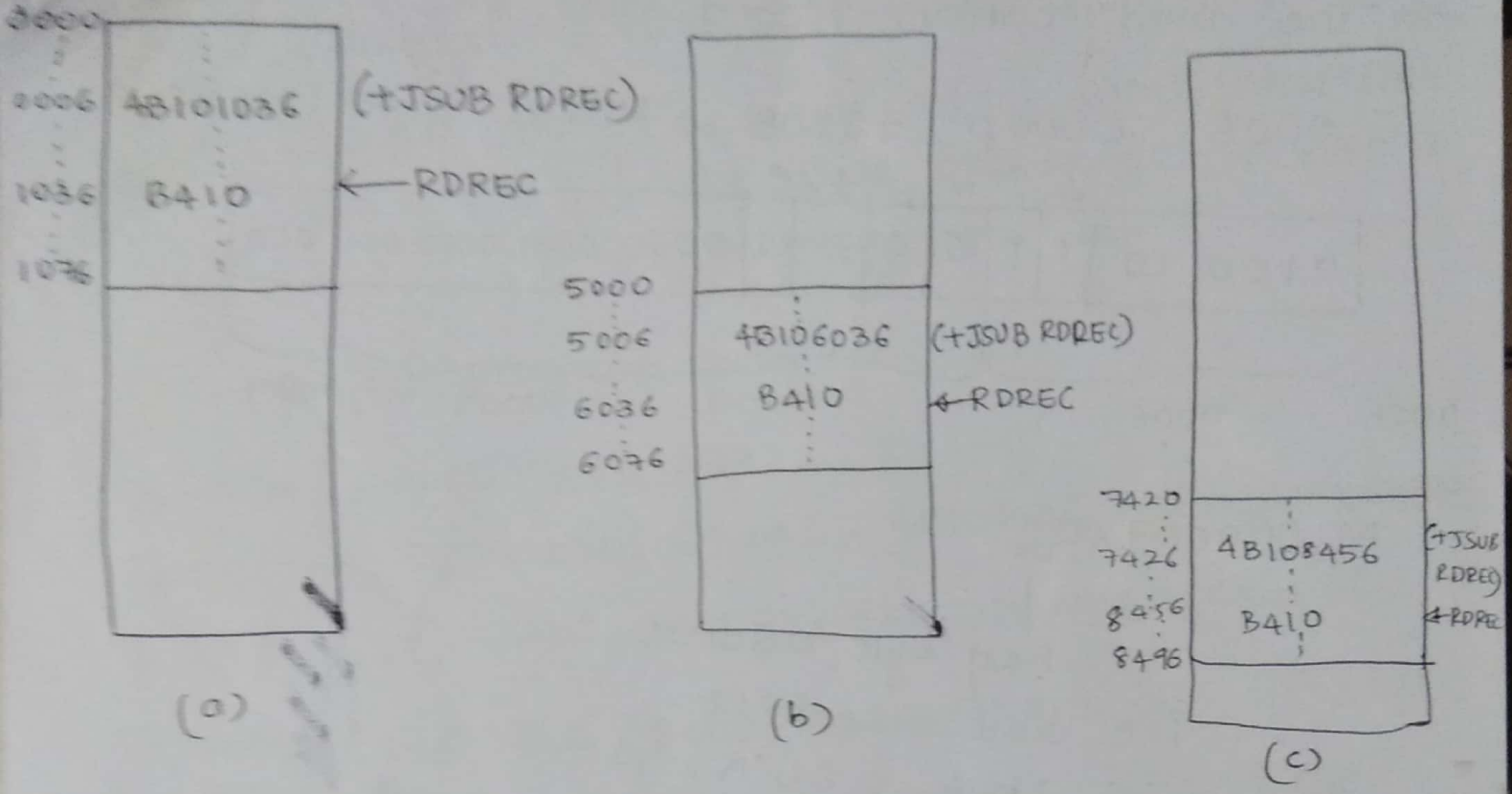
Ex: 20 bits = 5 half-bytes.

→ The starting location is the location of the byte containing the left most bits of the address field to be modified. If this field occupies an odd number of half-bytes, it is assumed to begin in the first byte at the starting location.

Ex: SIC/XE program.

```
5    0000  COPY   START   0
10   0000  FIRST  STL     RETADR    17202D
12   0003         LDB     #LENGTH   69202D
13                BASE    LENGTH
15   0006  CLOOP  +JSUB   RDREC     4B101036
                  :
                  :
35   0013         +JSUB   WRREC     4B10105D
40   0017         J       CLOOP     3F2FEC
                  :
                  :
65   0026         +JSUB   WRREC     4B10105D
:
105  0036  BUFFER RESB    4096
:
125  1036  RDREC  CLEAR   X         B410
```

→ Program is loaded at address 0000



→ The JSUB instruction at line 15 is loaded at address 0006, the address field contains 01036 (address of RDREC) is 0006 +JSUB RDREC 4B101036.

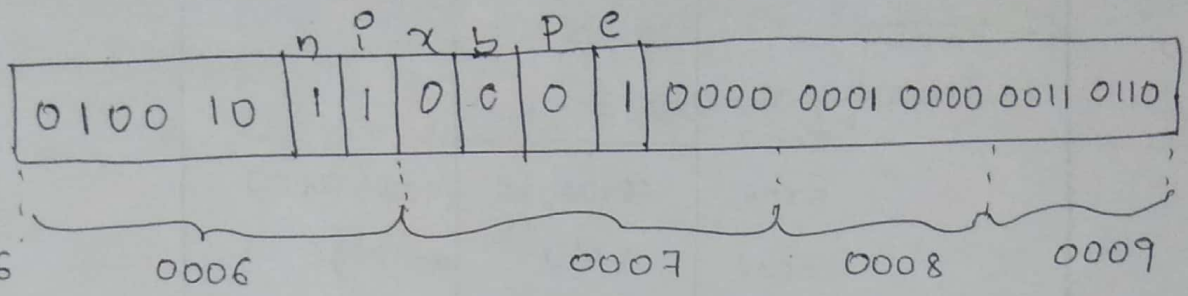
→ Suppose we want to load this program beginning at address 5000, as shown in fig (b), the address of instruction labeled RDREC will be 6036.

→ Likewise if we load at 7420 as in fig (c), then address of RDREC will be 4B108456.

→ It means, irrespective of the starting address loaded, RDREC is always 1036 bytes past starting address of the program. This is the reason we initialize the location counter to 0. (i.e., relative to the starting address).

→ The modification record looks like

0006 CLOOP +JSUB RDREC 4B101036



M^000007^05

↓

half byte address.

i.e add = 20 bits

1 byte = 8 bits.

1 half byte = 4 bits.

∴ 20 bits has 05 half bytes.

Actually at location 0007, first half byte is a part of flag bits x, b, p, e, But 05 tells loader to modify only last 5 half bytes i.e.) 4B1 remains unchanged.

Relocation for instruction of line 35 and 65

35	0013	+JSUB	WRREC	4B10105D
65	0026	+JSUB	WRREC	4B10105D

M^000014^05 & M^000027^05

→ If we add 5000 then address should be

4B1 01036
5000
4B1 06036

4B1 01036
7420
4B1 08456

→ Some instructions like

CLEAR S
LDA #3

} does not need modification because operands are not a memory address.

→ The object program written as.

H^COPY..^000000^001077
T^000000^1D^17202D^69202D^4B101036^032026^-4B101053^032010
T^00001D^13^0F2016^010003^0F200D^4B10105D..^454F46.
T^001036^1D^B410^B400^B440^75101000^....^B850
T^001053^1D^3B2FEA^134000^4F0000^....^B850
T^001070^07^3B2FEF^4F0000^05
M^000007^05
M^000014^05
M^000027^05
E^000000.

In General,

→ The only part of the program that require modification at load time are those that specify direct addresses.

→ The rest of the instructions need not be modified
  ↳ Not a memory address
  ↳ pc & Base relative.

→ from the object program, it is not possible to distinguish the address and constant.